

KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING · DEPARTMENT OF ELECTRICAL AND ELECTRONICS

TETE 3102: MICROCONTROLLERS AND APPLICATIONS

LABORATORY REPORT: TASK 4

Multi-Slave Bus Orchestration: Digital Clock & Temperature Monitor

Mukama Robert (26/U/ETE/21285/PE)
Nabudere Zerubabel (25/U/ETE/05465/PE)
Nanfuka Shakirah (25/U/ETE/05468/PE)
Kidega Wilkie Cephas (25/U/ETE/05460/PE)

Group Number: Group 4
Platforms: STM32F411RE & Arduino Uno
Course Supervisor: Dr. Mugerwa Dickson PhD

OBJECTIVE & LAB FOCUS

The objective of this laboratory task is to orchestrate two independent I^2C peripheral devicesan LCD1602 character display and a DS3231 temperature-compensated Real-Time Clock (RTC)sharing common SDA and SCL bus lines. Students implement register-level read/write transactions via repeated START conditions, parse Binary-Coded Decimal (BCD) time representations, extract high-resolution ambient temperature telemetry, and embed defensive bus error handling.

1 Multi-Slave Coexistence & Hardware Interfacing

Both the PCF8574 (LCD) and DS3231 (RTC) feature open-drain I/O stages connected in parallel to STM32 pins PB9 (SDA) and PB8 (SCL), or Arduino pins A4 and A5.

Table 1: Dual-Device Bus Interconnection Architecture

Device Node	7-bit Address	Write Byte	Read Byte	Internal Architecture / Role
LCD1602 (PCF8574)	`0x27`	`0x4E`	`0x4F`	Output display; receives 4-bit nibble commands.
DS3231 RTC	`0x68`	`0xD0`	`0xD1`	Register-based; provides BCD time and temperature.
Shared Bus Lines	SDA (PB9 / A4), SCL (PB8 / A5)			Pulled up via 2.2 kΩ to 4.7 kΩ resistors.

THEORETICAL FOUNDATION & PROTOCOL RULES

Repeated START Condition (S_r): Reading a specific register on the DS3231 requires a combined transaction. The master first issues a START, transmits the write address (`0xD0`), and sends the target register pointer (`0x00`). Instead of issuing a STOP condition (which would surrender bus ownership), the master immediately issues a **repeated START** (S_r) and broadcasts the read address (`0xD1`). This guarantees atomic bus control in multi-master environments.

2 Firmware Implementation

2.1 Platform 1: STM32 HAL Implementation (main.c)

The STM32 implementation uses `HAL_I2C_Mem_Write` and `HAL_I2C_Mem_Read` to interface with the DS3231 registers, and custom byte routines for the LCD.

Listing 1: STM32 HAL Digital Clock with Temperature & Error Handling

```c
#define DS3231_ADDR (0x68 << 1)
#define LCD_ADDR    (0x27 << 1)

static uint8_t bcd2dec(uint8_t v) { return (uint8_t)((v >> 4) * 10 + (v & 0x0F)); }
static uint8_t dec2bcd(uint8_t v) { return (uint8_t)(((v / 10) << 4) | (v % 10)); }

HAL_StatusTypeDef ds3231_set_time(uint8_t h, uint8_t m, uint8_t s) {
    uint8_t t[3] = { dec2bcd(s), dec2bcd(m), dec2bcd(h) };
    return HAL_I2C_Mem_Write(&hi2c1, DS3231_ADDR, 0x00, I2C_MEMADD_SIZE_8BIT, t, 3, 100);
}

HAL_StatusTypeDef ds3231_get_time(uint8_t *h, uint8_t *m, uint8_t *s) {
    uint8_t t[3];
    HAL_StatusTypeDef res = HAL_I2C_Mem_Read(&hi2c1, DS3231_ADDR, 0x00,
                                             I2C_MEMADD_SIZE_8BIT, t, 3, 100);
    if (res == HAL_OK) {
        *s = bcd2dec(t[0] & 0x7F);
        *m = bcd2dec(t[1]);
        *h = bcd2dec(t[2] & 0x3F); // 24-hour mode
    }
    return res;
}

/* Reads high-precision internal temperature registers 0x11 (MSB) and 0x12 (LSB) */
HAL_StatusTypeDef ds3231_get_temp(float *temp) {
    uint8_t buf[2];
    HAL_StatusTypeDef res = HAL_I2C_Mem_Read(&hi2c1, DS3231_ADDR, 0x11,
                                             I2C_MEMADD_SIZE_8BIT, buf, 2, 100);
    if (res == HAL_OK) {
        int8_t msb = (int8_t)buf[0];
        uint8_t lsb = buf[1] >> 6; // Upper two bits represent 0.25 deg C steps
        *temp = (float)msb + (lsb * 0.25f);
    }
    return res;
}

/* Main periodic execution loop */
void clock_application_loop(void) {
    uint8_t h, m, s;
    float temp = 0.0f;
    char line1[17], line2[17];

    while (1) {
        if (ds3231_get_time(&h, &m, &s) == HAL_OK && ds3231_get_temp(&temp) == HAL_OK) {
            snprintf(line1, sizeof(line1), "TIME:  %02d:%02d:%02d", h, m, s);
            snprintf(line2, sizeof(line2), "TEMP:  %5.2f C", temp);

            lcd_set_cursor(0, 0); lcd_print(line1);
            lcd_set_cursor(1, 0); lcd_print(line2);
        } else {
            /* Defensive bus recovery and fault notification */
            lcd_set_cursor(0, 0); lcd_print("** BUS ERROR ** ");
            lcd_set_cursor(1, 0); lcd_print("RTC NOT DETECTED");
        }
        HAL_Delay(1000);
    }
}
```

2.2 Platform 2: Arduino Uno Sketch (Wire Library)

Listing 2: Arduino Uno Dual-Device Clock Implementation

```ino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

const byte RTC_ADDR = 0x68;
LiquidCrystal_I2C lcd(0x27, 16, 2);

byte bcd2dec(byte v) { return (v >> 4) * 10 + (v & 0x0F); }
byte dec2bcd(byte v) { return ((v / 10) << 4) | (v % 10); }

bool readRTC(byte &h, byte &m, byte &s, float &temp) {
  Wire.beginTransmission(RTC_ADDR);
  Wire.write(0x00);
  if (Wire.endTransmission(false) != 0) return false; // Repeated START

  if (Wire.requestFrom((uint8_t)RTC_ADDR, (uint8_t)3) != 3) return false;
  s = bcd2dec(Wire.read() & 0x7F);
  m = bcd2dec(Wire.read());
  h = bcd2dec(Wire.read() & 0x3F);

  // Read temperature registers 0x11 and 0x12
  Wire.beginTransmission(RTC_ADDR);
  Wire.write(0x11);
  if (Wire.endTransmission(false) == 0 && Wire.requestFrom((uint8_t)RTC_ADDR,
      (uint8_t)2) == 2) {
    int8_t msb = (int8_t)Wire.read();
    uint8_t lsb = Wire.read() >> 6;
    temp = (float)msb + (lsb * 0.25f);
  }
  return true;
}

void setup() {
  Wire.begin();
  lcd.init();
  lcd.backlight();
}

void loop() {
  byte h, m, s;
  float temp = 0.0;
  if (readRTC(h, m, s, temp)) {
    char buf1[17], buf2[17];
    sprintf(buf1, "TIME: %02d:%02d:%02d", h, m, s);
    sprintf(buf2, "TEMP: %.2f C", temp);
    lcd.setCursor(0, 0); lcd.print(buf1);
    lcd.setCursor(0, 1); lcd.print(buf2);
  } else {
    lcd.setCursor(0, 0); lcd.print("RTC ERROR       ");
    lcd.setCursor(0, 1); lcd.print("RETRYING...     ");
  }
  delay(1000);
}
```

3 Mandatory Task Questions & Detailed Solutions

Task Question 1

Why can two devices share the physical bus lines without electrical interference?

Answer: Two devices can share the physical SDA and SCL lines because all I^2C output drivers are strictly **open-drain** (or open-collector). Neither the master nor any slave contains an active PMOS pull-up transistor to V_{DD}. Instead, the bus voltage is established passively by external pull-up resistors (R_p). Devices only assert

signals by driving an open-drain NMOS to pull the line to GND. When idle, all devices release the line into a high-impedance (*Hi-Z*) state, creating a non-destructive **wired-AND** electrical node.

Task Question 2

What stops the LCD and RTC from talking at the same time?

Answer: Two primary architectural mechanisms enforce strict serialization:

1. **Unique In-Band Addressing:** The protocol requires that every transaction begin with an address byte immediately following the START condition. Slaves continuously monitor the bus, but only the slave whose hardwired or pin-configured address matches (e.g., `0x27` for LCD, `0x68` for DS3231) acknowledges and participates. All non-addressed slaves remain silent with their open-drain outputs in high-impedance mode.

2. **Single-Master Serialization:** Because the microcontroller acts as the sole bus master, it exclusively controls the SCL clock and initiates transactions sequentially. The master talks to the RTC, terminates or transitions the frame, and subsequently initiates a separate transaction to the LCD.

Task Question 3

What is BCD (Binary-Coded Decimal), and what would be visually displayed on the LCD if you forgot to convert it?

Answer:

- **Definition of BCD:** Binary-Coded Decimal represents each decimal digit (0–9) as a distinct 4-bit nibble (0000_2–1001_2). For example, 45 seconds is stored as byte `0x45` ($0100\ 0101_2$), rather than the pure binary value $45_{10} = \texttt{0x2D} = 0010\ 1101_2$.

- **Display Glitches if Unconverted:** If the raw BCD byte `0x45` is printed as a standard decimal integer (‘

$$\text{Interpreted Decimal Value} = 4 \times 16 + 5 = \mathbf{69} \tag{1}$$

  The clock would display `"45"` as `"69"` seconds! Furthermore, at decimal second boundaries, the seconds count would jump unnaturally: from second 09 ($0000\ 1001_2 = 9$) directly to 16 ($0001\ 0000_2 = 16$), entirely skipping numbers 10 through 15.

4 Conclusion & Practical Engineering Observations

Task 4 validates the practical power of serial bus shared topologies: controlling both a high-accuracy real-time clock and a 32-character display using only two microcontroller I/O pins. The inclusion of defensive error routines ensures system resilience, gracefully alerting users to hardware disconnections without crashing the operating firmware.